CryptoMesh Logical Module Documentation

This document divides the application into logical modules for design,

ownership, and future maintainability. It does not require a physical Gradle

module split. The current app remains a single Android application module while

these boundaries describe how the code should be understood and extended.

Recent reliability and media-transfer updates

The most recent changes focused on three areas:

- Session resilience: stale authenticated sessions are removed after transport disconnects so peers do not remain falsely authenticated while the link is offline.
- Media transfer reliability: large media payloads are sent asynchronously in the background, fragments are reassembled before completion, and interrupted/incomplete transfers are marked as Failed instead of silently disappearing.
- User-visible transfer flow: queueing, loading states, and in-chat media cards were improved so the user can preview images/video/PDF items in the correct conversational order without blocking text chat.

For the UI layer, transfers are now treated as conversation timeline items rather than top-of-chat floating cards. This keeps normal messages above and below the media item in the correct order. For the repository layer, outgoing media transfer is decoupled from the immediate send action and continues through chunk fragmentation plus acknowledgement-driven completion.

Module Map

| Logical module | Primary files | Main responsibility |

| --- | --- | --- |

| App Shell | `CryptoMeshApplication.kt`, `MainActivity.kt` | Build app-wide dependencies and bridge Android lifecycle entry points into the app. |

| Navigation | `navigation/*` | Define routes, destinations, and top-level screen composition. |

| UI Screens | `ui/screens/*` | Render user-facing screens and collect direct user actions. |

| UI State | `ui/state/*` | Own screen state, view models, and UI-facing command methods. |

| Design System | `ui/components/*`, `ui/theme/*` | Provide reusable UI primitives and app styling. |

| Identity and Crypto | `crypto/*`, selected `protocol/*` | Manage local identity keys, signatures, handshakes, and packet encryption. |

| Wire Protocol | `protocol/*` | Define offline packet envelopes, secure packet codecs, and media transfer payloads. |

| BLE Transport | `transport/*` | Discover nearby devices and move framed bytes over Bluetooth Low Energy. |

| Persistence | `data/local/*` | Store local identity, secure packets, media transfers, and media chunks. |

| Domain Repositories | `data/repository/*` | Coordinate identity, packet storage, BLE sessions, relay behavior, and media transfer state. |

| Notifications | `notification/*` | Convert repository state changes into local Android notifications. |

| Tests | `app/src/test/java/com/cryptomesh/frontend/*` | Validate protocol, repository, state, notification, and frame behavior. |

1. App Shell

Files

- `app/src/main/java/com/cryptomesh/frontend/CryptoMeshApplication.kt`
- `app/src/main/java/com/cryptomesh/frontend/MainActivity.kt`

Responsibility

The App Shell is the application composition root. It creates singleton instances for storage, repositories, BLE transport, cryptographic services, and notification coordination.

`CryptoMeshApplication` owns the `AppContainer`, which wires:

- Room database.
- Android-backed identity keystore.
- Identity, packet, media transfer, and direct mesh repositories.
- Local media file storage.
- Android BLE transport.
- Notification coordinator.

`MainActivity` owns Android activity concerns:

- Compose content setup.
- Runtime notification permission checks.
- Launching the app from notification intents.
- Passing the app container into the UI tree.

Boundary Rules

- App-wide dependencies should be created here, not directly inside screens.

• Android lifecycle entry points should stay in this layer.

• Repositories should remain injectable through the app container so tests can replace them with fakes.

2. Navigation

Files

• `app/src/main/java/com/cryptomesh/frontend/navigation/AppRoute.kt`

• `app/src/main/java/com/cryptomesh/frontend/navigation/CryptoMeshApp.kt`

• `app/src/main/java/com/cryptomesh/frontend/navigation/MainDestination.kt`

Responsibility

The Navigation module defines the top-level app structure. It decides which screen is shown, how bottom navigation works, and how screen-level view models are created from repository dependencies.

Main routes include:

• Welcome and identity creation.

• Dashboard.

• Peer discovery.

• Chat.

• Profile.

• Permissions.

Boundary Rules

• Navigation may construct view models from app container dependencies.

• Navigation should not contain business rules for BLE sessions, packet routing, media transfer assembly, or persistence.

• Screens should receive only the state and callbacks they need.

3. UI Screens

Files

- `app/src/main/java/com/cryptomesh/frontend/ui/screens/WelcomeScreen.kt`
- `app/src/main/java/com/cryptomesh/frontend/ui/screens/CreateIdentityScreen.kt`
- `app/src/main/java/com/cryptomesh/frontend/ui/screens/DashboardScreen.kt`
- `app/src/main/java/com/cryptomesh/frontend/ui/screens/PeersScreen.kt`
- `app/src/main/java/com/cryptomesh/frontend/ui/screens/ChatScreen.kt`
- `app/src/main/java/com/cryptomesh/frontend/ui/screens/PermissionsScreen.kt`
- `app/src/main/java/com/cryptomesh/frontend/ui/screens/ProfileScreen.kt`

Responsibility

UI Screens render the app experience and collect user intent. They should be stateless where possible, with mutable behavior delegated to view models.

Screen responsibilities:

- `WelcomeScreen`: first-run entry point.
- `CreateIdentityScreen`: creates a local offline identity.
- `DashboardScreen`: shows mesh status, peer count, and message activity.
- `PeersScreen`: displays discovered peers and scan/advertise controls.
- `ChatScreen`: displays offline messages and media transfer cards in timeline order, and allows sending text, photos, videos, audio, and PDF payloads through BLE.

Recent UI behavior:

- media cards show a loading/progress state while transmitting
- previews render for completed image/video/PDF assets in the same thread as normal messages
- messages before and after media are ordered chronologically rather than pinned to the top of the chat thread
- completed media can be opened from the app using the system viewer or compatible device apps
- `PermissionsScreen`: explains and requests required local Android permissions.
- `ProfileScreen`: displays local identity and reset controls.

Boundary Rules

• Screens should not talk directly to DAOs, BLE transport, crypto services, or wire codecs.

• Screens call view-model methods such as send message, send media, start scanning, or reset identity.

• File and media picker results are passed to view models as Android `Uri` values; transfer processing belongs below the UI layer.

4. UI State

Files

• `app/src/main/java/com/cryptomesh/frontend/ui/state/ChatUiState.kt`
• `app/src/main/java/com/cryptomesh/frontend/ui/state/ChatViewModel.kt`
• `app/src/main/java/com/cryptomesh/frontend/ui/state/CryptoMeshViewModel.kt`
• `app/src/main/java/com/cryptomesh/frontend/ui/state/LocalIdentity.kt`
• `app/src/main/java/com/cryptomesh/frontend/ui/state/PeerDiscoveryUiState.kt`
• `app/src/main/java/com/cryptomesh/frontend/ui/state/PeerDiscoveryViewModel.kt`

Responsibility

The UI State module adapts domain repository state into Compose-friendly state.
It owns user-facing operations and keeps screens thin.

Key view models:

• `CryptoMeshViewModel`: observes local identity, creates identity, resets identity, and exposes global app state.

• `PeerDiscoveryViewModel`: observes BLE mesh state and controls discovery.

• `ChatViewModel`: observes packet and media transfer state, sends text, and sends media selected from the device.

Boundary Rules

• View models may depend on repositories.
• View models should expose immutable UI state to screens.
• View models should avoid Android transport, database, or cryptographic

implementation details.

5. Design System

Files

- `app/src/main/java/com/cryptomesh/frontend/ui/components/ActionButton.kt`
- `app/src/main/java/com/cryptomesh/frontend/ui/components/EmptyState.kt`
- `app/src/main/java/com/cryptomesh/frontend/ui/components/InfoRow.kt`
- `app/src/main/java/com/cryptomesh/frontend/ui/components/MetricCard.kt`
- `app/src/main/java/com/cryptomesh/frontend/ui/components/ScreenHeader.kt`
- `app/src/main/java/com/cryptomesh/frontend/ui/components/SectionHeader.kt`
- `app/src/main/java/com/cryptomesh/frontend/ui/components/StatusPill.kt`
- `app/src/main/java/com/cryptomesh/frontend/ui/theme/Color.kt`
- `app/src/main/java/com/cryptomesh/frontend/ui/theme/Theme.kt`
- `app/src/main/java/com/cryptomesh/frontend/ui/theme/Type.kt`

Responsibility

The Design System module keeps visual primitives consistent across screens. It contains shared components for headers, metrics, status labels, empty states, and app theme tokens.

Boundary Rules

- Components should stay reusable and screen-agnostic.
- Components should not initiate BLE operations, database work, media transfer work, or navigation side effects.
- Theme files should contain styling tokens and Material theme setup only.

6. Identity and Crypto

Files

- `app/src/main/java/com/cryptomesh/frontend/crypto/AndroidIdentityKeyStore.kt`
- `app/src/main/java/com/cryptomesh/frontend/crypto/PacketSignatureService.kt`

- `app/src/main/java/com/cryptomesh/frontend/crypto/SessionCipher.kt`
- `app/src/main/java/com/cryptomesh/frontend/protocol/AuthenticatedHandshake.kt`
- `app/src/main/java/com/cryptomesh/frontend/protocol/SecurePacketCodec.kt`
- `app/src/main/java/com/cryptomesh/frontend/protocol/SecurePacketEnvelope.kt`

Responsibility

The Identity and Crypto module protects local identity and message integrity. It provides:

- Android Keystore-backed P-256 identity key creation and retrieval.
- Packet signing and verification.
- Authenticated handshake support.
- Session encryption and decryption.
- Secure packet serialization and parsing.

The app is designed for offline trust establishment over BLE. Secure packets are sealed before transport and can be stored or relayed without requiring an internet backend.

Boundary Rules

- Private key material stays behind the identity keystore abstraction.
- Transport code should not inspect or modify encrypted packet payloads.
- UI code should never handle raw private keys, session keys, or packet signatures.

7. Wire Protocol

Files

- `app/src/main/java/com/cryptomesh/frontend/protocol/DirectWireProtocol.kt`
- `app/src/main/java/com/cryptomesh/frontend/protocol/MediaTransferPayloads.kt`
- `app/src/main/java/com/cryptomesh/frontend/protocol/SecurePacketEnvelope.kt`
- `app/src/main/java/com/cryptomesh/frontend/protocol/SecurePacketCodec.kt`
- `app/src/main/java/com/cryptomesh/frontend/protocol/AuthenticatedHandshake.kt`

Responsibility

The Wire Protocol module defines the application-level language used between offline peers.

Protocol layers:

- `DirectWireProtocol`: outer BLE wire envelopes for handshake and secure packet delivery.
- `SecurePacketEnvelope`: authenticated packet metadata, routing information, timestamps, signatures, and payload type.
- `SecurePacketCodec`: canonical encoding and decoding for secure packets.
- `MediaTransferPayloads`: structured payloads for media offer, chunk, and completion packets.
- `AuthenticatedHandshake`: peer authentication and session bootstrap.

Packet categories include:

- Text message.
- Acknowledgement.
- Media offer.
- Media chunk.
- Media complete.

Boundary Rules

- Protocol classes should remain deterministic and testable.
- Protocol code should avoid Android UI and storage dependencies.
- New packet types should be added here first, then handled by repositories and UI state.

8. BLE Transport

Files

- `app/src/main/java/com/cryptomesh/frontend/transport/NearbyTransport.kt`
- `app/src/main/java/com/cryptomesh/frontend/transport/AndroidBleTransport.kt`

- `app/src/main/java/com/cryptomesh/frontend/transport/BleFrameCodec.kt`
- `app/src/main/java/com/cryptomesh/frontend/transport/BluetoothPermissions.kt`

Responsibility

The BLE Transport module is the app's offline physical transport. It discovers nearby peers, advertises local availability, establishes direct BLE sessions, and moves framed bytes between Android devices.

Transport responsibilities:

- Runtime Bluetooth permission checks.
- BLE advertising.
- BLE scanning.
- GATT service and characteristic behavior.
- Byte frame encoding and decoding.
- Peer connection state.

Boundary Rules

- Transport moves bytes and reports connection events.
- Transport should not own message persistence, relay decisions, media reassembly, or UI state.
- The app should remain functional without `android.permission.INTERNET`.

9. Persistence

Files

- `app/src/main/java/com/cryptomesh/frontend/data/local/CryptoMeshDatabase.kt`
- `app/src/main/java/com/cryptomesh/frontend/data/local/IdentityDao.kt`
- `app/src/main/java/com/cryptomesh/frontend/data/local/LocalIdentityEntity.kt`
- `app/src/main/java/com/cryptomesh/frontend/data/local/SecurePacketDao.kt`
- `app/src/main/java/com/cryptomesh/frontend/data/local/SecurePacketEntity.kt`
- `app/src/main/java/com/cryptomesh/frontend/data/local/MediaTransferDao.kt`
- `app/src/main/java/com/cryptomesh/frontend/data/local/MediaTransferEntity.kt`

- `app/src/main/java/com/cryptomesh/frontend/data/local/MediaChunkDao.kt`
- `app/src/main/java/com/cryptomesh/frontend/data/local/MediaChunkEntity.kt`

Responsibility

The Persistence module stores durable offline state with Room.

Tables:

- `local_identity`: current local offline identity.
- `secure_packets`: signed, encrypted, store-carry-forward packets.
- `media_transfers`: media transfer metadata, progress, status, and file output path.
- `media_chunks`: individual media chunks used for BLE transfer and reassembly.

Schema version 2 adds media transfer persistence. The version 1 to 2 migration creates the media transfer and chunk tables without requiring a network service.

Boundary Rules

- DAOs expose storage operations only.
- Domain repositories decide how stored rows are interpreted.
- UI and transport should not access DAOs directly.

10. Domain Repositories

Files

- `app/src/main/java/com/cryptomesh/frontend/data/repository/IdentityRepository.kt`
- `app/src/main/java/com/cryptomesh/frontend/data/repository/SecurePacketRepository.kt`
- `app/src/main/java/com/cryptomesh/frontend/data/repository/DirectMeshRepository.kt`
- `app/src/main/java/com/cryptomesh/frontend/data/repository/MediaTransferRepository.kt`
- `app/src/main/java/com/cryptomesh/frontend/data/repository/MediaTransferEngine.kt`
- `app/src/main/java/com/cryptomesh/frontend/data/repository/MediaFileStore.kt`

Responsibility

Domain Repositories are the business logic layer. They coordinate persistence, protocol encoding, cryptography, BLE transport, and UI-facing state.

Repository responsibilities:

- `IdentityRepository`: create, observe, and reset local identities.
- `SecurePacketRepository`: store secure packets, track delivery status, and expose packet history.
- `DirectMeshRepository`: orchestrate peer discovery, direct BLE sessions, message sending, packet receiving, acknowledgement handling, and store-carry-forward relay.
- `MediaTransferRepository`: store media transfer metadata and chunks.
- `MediaTransferEngine`: prepare local media into encrypted chunks and rebuild received media from chunks.
- `MediaFileStore`: copy picked media into app-owned storage and persist reconstructed files.

Boundary Rules

- Repositories are the only layer that should combine transport, protocol, storage, and crypto decisions.
- Relay behavior belongs in `DirectMeshRepository`.
- Media chunking and reassembly belong in `MediaTransferEngine`.
- File-system media storage belongs behind `MediaFileStore`.

11. Notifications

Files

- `app/src/main/java/com/cryptomesh/frontend/notification/AndroidMeshNotifier.kt`
- `app/src/main/java/com/cryptomesh/frontend/notification/MeshNotificationCoordinator.kt`
- `app/src/main/java/com/cryptomesh/frontend/notification/MeshNotificationEvent.kt`

Responsibility

The Notifications module observes mesh state and emits local Android

notifications for useful user-facing events.

Notification events include:

- New offline message received.
- Peer discovered.
- Peer connected.
- Peer disconnected.
- Media transfer progress.
- Media transfer completion.
- Media transfer failure.

Boundary Rules

- Notification logic should observe repository state instead of owning message or media transfer state.
- Notification rendering belongs in `AndroidMeshNotifier`.
- Event detection belongs in `MeshNotificationEvent` and the coordinator.

12. Tests

Files

- `app/src/test/java/com/cryptomesh/frontend/protocol/*`
- `app/src/test/java/com/cryptomesh/frontend/transport/*`
- `app/src/test/java/com/cryptomesh/frontend/data/repository/*`
- `app/src/test/java/com/cryptomesh/frontend/notification/*`
- `app/src/test/java/com/cryptomesh/frontend/ui/state/*`

Responsibility

The Tests module verifies behavior at the boundaries where regressions are most likely:

- Protocol encoding, decoding, and authenticated handshake behavior.
- BLE frame encoding and decoding.
- Identity and secure packet repository behavior.

- Direct BLE mesh repository behavior.
- Media transfer chunking, persistence, and reconstruction behavior.
- Notification event detection.
- View-model state mapping and user operations.

Boundary Rules

- Protocol tests should avoid Android runtime dependencies where possible.
- Repository tests should use fakes or in-memory storage when practical.
- UI state tests should verify state transitions rather than Compose rendering.

Runtime Flows

Identity Creation

1. `CreateIdentityScreen` collects alias information.
2. `CryptoMeshViewModel` calls `IdentityRepository`.
3. `IdentityRepository` creates a local identity and uses `AndroidIdentityKeyStore` for key material.
4. Identity metadata is persisted in Room.
5. Navigation moves into the main app once identity state is available.

Peer Discovery and Session Setup

1. `PeersScreen` requests scan or advertise behavior through `PeerDiscoveryViewModel`.
2. `PeerDiscoveryViewModel` calls `DirectMeshRepository`.
3. `DirectMeshRepository` starts BLE behavior through `NearbyTransport`.
4. `AndroidBleTransport` discovers or accepts a nearby peer.
5. `AuthenticatedHandshake` establishes an authenticated session.
6. `SessionCipher` protects packets sent inside the session.

Offline Text Message

1. `ChatScreen` sends text through `ChatViewModel`.
2. `ChatViewModel` calls `DirectMeshRepository`.

3. `DirectMeshRepository` creates a secure packet through protocol and crypto services.
4. `SecurePacketRepository` stores the packet for durable offline delivery.
5. If a session is active, BLE transport sends the packet immediately.
6. If no session is active, the packet remains queued for store-carry-forward delivery.
7. Receiving peers store, display, acknowledge, or relay the packet depending on destination and delivery state.

Offline Media Transfer

1. `ChatScreen` launches Android document selection for image, video, or audio.
2. `ChatViewModel` passes the selected `Uri` and media type to `DirectMeshRepository`.
3. `MediaFileStore` copies the selected file into app-owned local storage.
4. `MediaTransferEngine` splits the file into encrypted BLE-sized chunks.
5. `MediaTransferRepository` persists transfer metadata and chunk rows.
6. `DirectMeshRepository` sends a media offer packet followed by media chunk packets.
7. Receiving devices persist incoming chunks.
8. Once all chunks arrive, `MediaTransferEngine` reassembles and decrypts the file into local app storage.
9. UI state updates media progress and completion status.
10. Notifications report progress, completion, or failure.

Store-Carry-Forward Relay

1. Secure packets are stored durably when created or received.
2. When a peer connects, `DirectMeshRepository` evaluates stored packets that are still eligible for forwarding.
3. Packets destined for the connected peer, or packets that should be relayed, are sent over the active BLE session.
4. Relay preserves the sealed packet payload instead of decrypting or modifying message contents.
5. Acknowledgements update local delivery status.

Identity Reset

1. `ProfileScreen` requests reset through `CryptoMeshViewModel`.
2. `CryptoMeshViewModel` delegates to `IdentityRepository`.
3. Local identity metadata and key material are cleared.
4. Dependent local state is reset according to repository rules.
5. The app returns to first-run identity creation.

Logical Dependency Graph

Future Physical Split Option

If the app later needs Gradle-level module separation, the current logical modules can map cleanly to:

- `:app`: application shell and Android entry points.
- `:core-protocol`: secure packet and wire protocol models.
- `:core-crypto`: key storage, signatures, handshakes, and session cipher.
- `:data-local`: Room database, DAOs, and entities.
- `:transport-ble`: BLE transport and frame codec.
- `:domain-mesh`: repositories, relay behavior, and media transfer engine.
- `:feature-chat`: chat UI and chat view model.
- `:feature-peers`: peer discovery UI and view model.
- `:feature-profile`: identity/profile UI and view model.
- `:notifications`: local notification event detection and rendering.
- `:testing`: shared test fakes and helpers.

That split should only be done when build times, ownership, or reuse justify the extra complexity. For now, logical boundaries are enough.

Extension Rules

- Add new offline packet types in the protocol module first.
- Keep packet storage changes in the persistence and repository modules.

- Keep BLE-specific behavior inside the transport module.
- Keep encryption, signatures, and key handling out of UI code.
- Keep media file copy, chunking, and reconstruction out of screens.
- Keep notification rendering separate from event detection.
- Preserve the no-internet architecture; peer communication must remain BLE based and offline.